COMPUTER NETWORKS — UE24CS252B

SDN Mininet Based Simulation Project

SDN-Based Access Control System

Name: Aanya K
SRN: PES2UG24CS011

1. Problem Statement

This project implements an SDN-based firewall using Mininet and the Ryu OpenFlow controller. The system maintains a whitelist of authorized MAC addresses and installs explicit allow/deny OpenFlow flow rules to permit only whitelisted hosts to communicate, while blocking all unauthorized hosts.

The project demonstrates:

- Controller-switch interaction using the OpenFlow protocol
- Flow rule design using match-action pairs with priorities and timeouts
- Network behavior observation using Wireshark and iperf
- Access control enforcement at Layer 2 using MAC-based filtering

2. Objective

- Maintain a whitelist of authorized MAC addresses
- Install explicit allow/deny OpenFlow flow rules on the switch
- Block unauthorized hosts from communicating with any host in the network
- Verify access control through two test scenarios and regression testing
- Measure and interpret network performance metrics (latency, throughput, flow table stats)

3. Theory & Background

3.1 Software Defined Networking (SDN)

Software Defined Networking (SDN) is a network architecture that separates the control plane from the data plane. In traditional networks, each switch makes its own forwarding decisions independently. In SDN, a single centralized controller makes all decisions and programs all switches with rules.

Component	Traditional Network	SDN
Decision Maker	Each switch independently	Central Ryu controller
Rule Configuration	Manual on each device	Programmatic via OpenFlow
Global View	None	Controller sees entire network
Flexibility	Low — hardware configs	High — software changes only

- Control Plane: Where decisions are made. In this project = Ryu controller
- Data Plane: Where packets are forwarded. In this project = Open vSwitch s1
- Southbound Interface: Protocol between controller and switch. In this project = OpenFlow 1.3

3.2 OpenFlow Protocol

OpenFlow is the protocol used by the SDN controller to install flow rules into the switch. Every flow rule has three parts:

- Match: Which packets does this rule apply to? Fields: eth_src (source MAC), eth_dst (destination MAC), in_port (input port)
- Action: What to do with matching packets? Options: output to a port, flood all ports, or drop (empty action list = drop)
- Priority: Which rule wins when multiple rules match? Higher number = higher priority

When a packet arrives at the switch with no matching flow rule, it triggers a packet_in event — the packet is sent to the controller. The controller processes it and typically installs a new flow rule so future similar packets are handled by the switch directly, without controller involvement.

3.3 Mininet

Mininet is a network emulator that creates a realistic virtual network on a single Linux machine. It uses Linux network namespaces to create isolated virtual hosts. Hosts are connected to an Open vSwitch (OVS) using virtual Ethernet (veth) pairs. Because Mininet runs real Linux kernel code, behavior closely matches real physical networks.

- Virtual hosts (h1–h4): Real Linux processes each with their own network interface
- Virtual switch (s1): Open vSwitch supporting OpenFlow 1.3
- Virtual links: veth pairs connecting each host to the switch
- sudo mn -c: Cleans up all Mininet state between sessions

3.4 Ryu Controller

Ryu is a Python-based SDN controller framework. A Ryu application is a Python class inheriting from RyuApp. Event handlers decorated with @set_ev_cls respond to OpenFlow events:

- EventOFPSwitchFeatures: Fires when a switch connects. Used to install the table-miss rule.
- EventOFPPacketIn: Fires when the switch sends a packet to the controller. This is where whitelist logic runs.
- OFPFlowMod: OpenFlow message sent to the switch to install a flow rule.
- OFPPacketOut: OpenFlow message sent to the switch to forward a specific packet.

3.5 MAC Address Based Access Control

MAC (Media Access Control) addresses are Layer 2 hardware-level identifiers assigned to network interfaces. They are unique per device and reliable identifiers within a LAN. MAC-based filtering was chosen because:

- MAC addresses operate at Layer 2, before IP routing — blocking happens at the lowest possible level
- They are fixed at the hardware level, making them reliable identifiers
- OpenFlow natively supports matching on eth_src and eth_dst fields

- IP addresses can be changed or spoofed, but MAC addresses cannot be easily changed on a fixed device

3.6 Flow Rule Priorities and Idle Timeouts

Priority	Rule Type	Purpose
0	Table-miss	Catches all packets with no matching rule — sends to controller
5	Allow rule	Forwards packets between whitelisted host pairs
10	Deny rule	Drops packets from blocked source MACs — overrides everything

Idle timeout means the rule is automatically removed from the switch if no matching packets arrive within that time. This keeps the flow table clean and prevents stale rules.

- Deny rules: 30 second idle timeout — short, because blocked hosts should not persist
- Allow rules: 60 second idle timeout — longer, for stable whitelisted sessions

3.7 Learning Switch

A learning switch dynamically builds a MAC-to-port table. When h1 sends a packet, the controller records that h1's MAC arrived on port 1. The next time a packet is destined for h1, the controller sends it directly to port 1 instead of flooding all ports. This reduces unnecessary broadcast traffic in the network.

4. Topology Design & Justification

4.1 Network Topology

One switch (s1) connected to four hosts in a star topology. The Ryu RemoteController runs on 127.0.0.1:6633 and connects to s1 via OpenFlow 1.3.

Component	Details
Switch	s1 — Open vSwitch, OpenFlow 1.3
Whitelisted Host 1	h1 — MAC: 00:00:00:00:00:01 — IP: 10.0.0.1
Whitelisted Host 2	h2 — MAC: 00:00:00:00:00:02 — IP: 10.0.0.2
Blocked Host 1	h3 — MAC: 00:00:00:00:00:03 — IP: 10.0.0.3
Blocked Host 2	h4 — MAC: 00:00:00:00:00:04 — IP: 10.0.0.4
Controller	Ryu RemoteController — 127.0.0.1:6633
OpenFlow Version	OpenFlow 1.3

4.2 Design Justification

A single switch star topology was chosen to keep the focus on access control logic without added complexity of multi-hop routing. MAC-based filtering was chosen because MAC addresses are hardware-level identifiers at Layer 2 — the most reliable way to identify hosts within a LAN. Using two whitelisted and two blocked hosts provides a clean demonstration of both allowed and denied communication, with room for a regression test on the second blocked host.

5. SDN Logic & Flow Rule Implementation

5.1 Whitelist

```
WHITELIST = {
    '00:00:00:00:00:01',  # h1 - authorized
    '00:00:00:00:00:02',  # h2 - authorized
}
```

5.2 Controller Logic

- switch_features_handler: Installs table-miss rule (priority 0) when switch connects. Ensures all unmatched packets go to controller.
- add_flow helper: Sends OFPFlowMod to switch with given match, action, priority, idle timeout.
- packet_in_handler: Checks source MAC against whitelist. Drops and installs deny rule (priority 10, idle 30s) for blocked hosts. For whitelisted hosts, learns port, installs allow rule (priority 5, idle 60s), forwards packet.

5.3 Flow Rule Design Table

Priority	Match Fields	Action	Idle Timeout	Purpose
0	All packets	Send to controller	None (permanent)	Table-miss — catch unmatched
5	in_port + eth_src + eth_dst	Output to learned port	60 seconds	Allow whitelisted pairs
10	eth_src (blocked MAC)	Drop (empty actions)	30 seconds	Block unauthorized sources

6. Setup & Execution

6.1 Installation Commands

```
sudo apt install mininet -y
pip3 install ryu
sudo apt install wireshark iperf3 -y
```

6.2 Running the Project

Terminal 1 — Start Ryu controller:

```
ryu-manager access_control.py
```

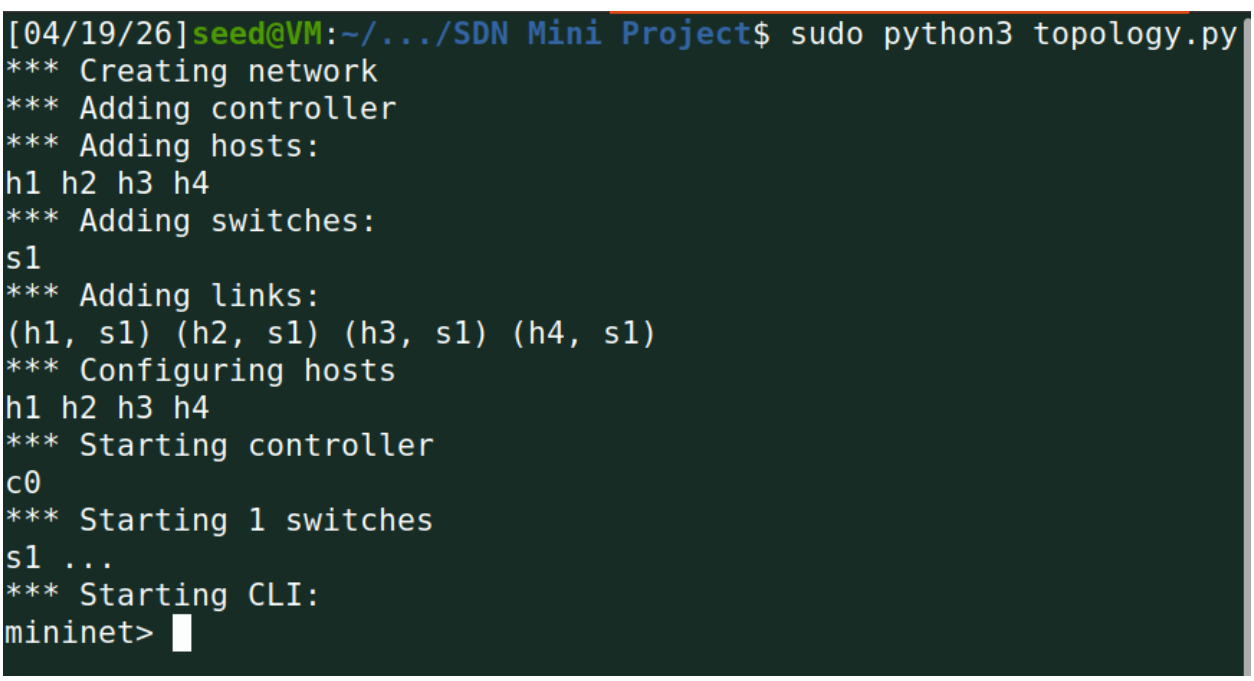
Terminal 2 — Start Mininet topology:

```
sudo python3 topology.py
```

7. Test Scenarios & Results

7.1 Topology Startup

Mininet creates the network with h1, h2, h3, h4 connected to s1. The switch connects to the Ryu controller on port 6633.



7.2 Scenario A — Allowed Communication (h1 ping h2)

Both h1 and h2 are whitelisted. The controller allows traffic and installs a priority-5 flow rule. The first ping is slower due to the initial packet_in round-trip; subsequent pings match the installed rule directly.

Parameter	Value
Command	h1 ping -c 3 h2
Packets Transmitted	3
Packets Received	3
Packet Loss	0%
Min RTT	0.109 ms
Avg RTT	1.578 ms
Max RTT	4.20 ms
Result	PASSED — whitelisted hosts communicate successfully

```
mininet> h1 ping -c 3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=4.20 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.424 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.109 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2018ms
rtt min/avg/max/mdev = 0.109/1.578/4.201/1.859 ms
mininet>
```

7.3 Scenario B — Blocked Communication (h3 ping h1)

h3 is not in the whitelist. The controller receives the packet_in, logs a BLOCKED warning, installs a priority-10 drop rule for h3's MAC, and returns without forwarding. All subsequent packets from h3 are dropped at the switch level.

Parameter	Value
Command	h3 ping -c 3 h1
Packets Transmitted	3
Packets Received	0
Packet Loss	100%
Error Message	Destination Host Unreachable
Ryu Log	BLOCKED: src=00:00:00:00:00:03 is not in whitelist. Dropping.
Result	PASSED — unauthorized host successfully blocked

```
mininet> h3 ping -c 3 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable
From 10.0.0.3 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, ti
me 2035ms
pipe 3
mininet>
```

7.4 Regression Test (h4 ping h2)

h4 is also not whitelisted. This test verifies that the access control policy applies consistently across all unauthorized hosts, not just h3.

Parameter	Value
Command	h4 ping -c 3 h2
Packets Transmitted	3
Packets Received	0
Packet Loss	100%
Error Message	Destination Host Unreachable
Result	PASSED — policy consistent across all blocked hosts

```
mininet> h4 ping -c 3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.4 icmp_seq=1 Destination Host Unreachable
From 10.0.0.4 icmp_seq=2 Destination Host Unreachable
From 10.0.0.4 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, ti
me 2035ms
pipe 3
mininet>
```

7.5 Flow Table

The ovs-ofctl dump-flows command shows all flow rules currently installed in the switch by the controller. The table-miss rule at priority=0 shows 33 packets and 2450 bytes processed — all the packets that required controller decision-making during the session.

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=240.887s, table=0, n_packets=33, n_bytes=245
0, priority=0 actions=CONTROLLER:65535
mininet>
```

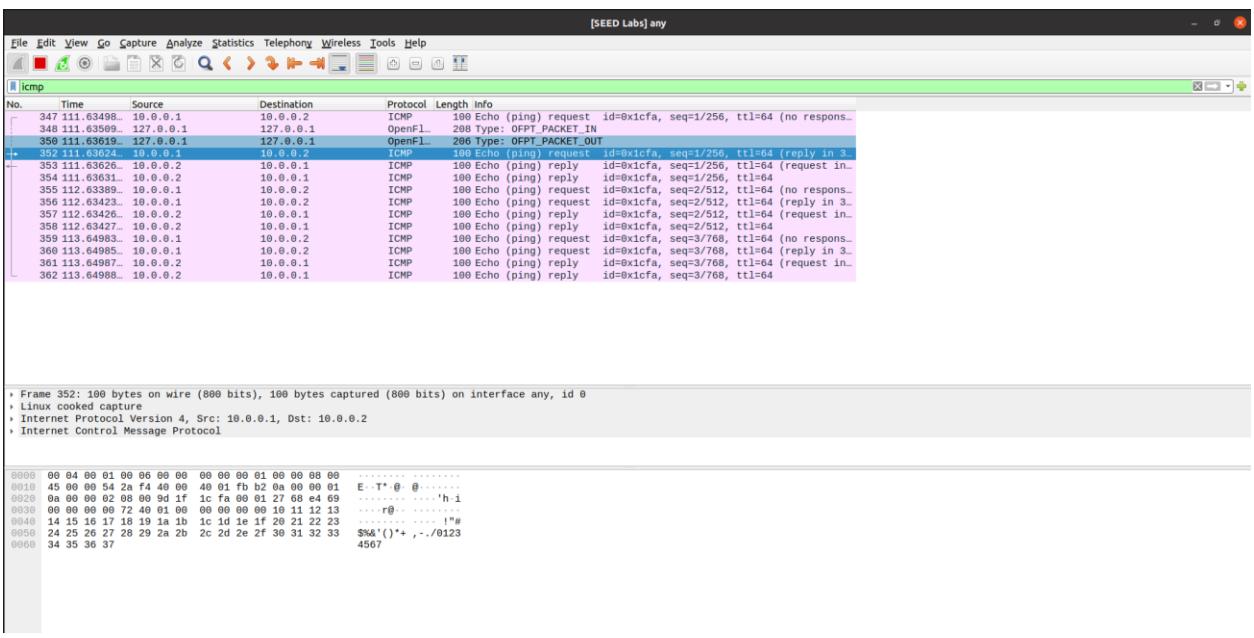
7.6 iperf Throughput

TCP bandwidth test between whitelisted hosts h1 and h2. Results confirm high throughput for authorized communication. Blocked hosts cannot run iperf as their packets are dropped at the switch before reaching any destination.

```
mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['7.71 Gbits/sec', '7.77 Gbits/sec']
mininet>
```

7.7 Wireshark — Allowed Traffic

Wireshark capture with icmp filter applied. Shows h1 (10.0.0.1) and h2 (10.0.0.2) exchanging ICMP requests and replies. OpenFlow PACKET_IN and PACKET_OUT messages are also visible, confirming active controller-switch interaction for each new flow.



7.8 Ryu Controller Logs — Blocked Traffic

The Ryu controller terminal shows BLOCKED warning messages for h3 and h4. This confirms the controller is actively monitoring all packet_in events and enforcing the whitelist policy throughout the session.

```
BLOCKED: dst=33:33:00:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:02 in_po
rt=1
BLOCKED: dst=33:33:00:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:04 dst=33:33:00:00:00:02 in_po
rt=4
BLOCKED: src=00:00:00:00:00:04 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:03 dst=33:33:00:00:00:02 in_po
rt=3
BLOCKED: src=00:00:00:00:00:03 is not in whitelist. Dropping.
```

```
[04/19/26]seed@VM:~/.../SDN Mini Project$ ryu-manager access_control.py
loading app access_control.py
loading app ryu.controller.ofp_handler
instantiating app access_control.py of AccessControlSwitch
instantiating app ryu.controller.ofp_handler of OFPHandler
Packet in dpid=1 src=00:00:00:00:00:03 dst=33:33:ff:00:00:03 in_port=3
BLOCKED: src=00:00:00:00:00:03 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:04 dst=33:33:ff:00:00:04 in_port=4
BLOCKED: src=00:00:00:00:00:04 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:00:00:00:16 in_port=2
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:16 in_port=1
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:ff:00:00:02 in_port=2
BLOCKED: dst=33:33:ff:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:ff:00:00:01 in_port=1
BLOCKED: dst=33:33:ff:00:00:01 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:00:00:00:16 in_port=2
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:00:00:00:02 in_port=2
BLOCKED: dst=33:33:00:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:16 in_port=1
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:02 in_port=1
BLOCKED: dst=33:33:00:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:00:00:00:16 in_port=2
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:16 in_port=1
BLOCKED: dst=33:33:00:00:00:16 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:01 dst=33:33:00:00:00:02 in_port=1
BLOCKED: dst=33:33:00:00:00:02 is not in whitelist. Dropping.
Packet in dpid=1 src=00:00:00:00:00:02 dst=33:33:00:00:00:02 in_port=2
```

8. Performance Observation & Analysis

8.1 Latency Analysis

Ping RTT between h1 and h2 shows an average of 1.578 ms. The first ping took 4.20 ms because the packet had no matching flow rule, triggering a packet_in event — the switch sent it to the controller, the controller processed it, installed a rule, and forwarded it. This round-trip adds delay. Subsequent pings were much faster (0.109 ms) because they matched the installed flow rule and were handled directly by the switch without controller involvement. This clearly shows the benefit of flow rule installation.

8.2 Throughput Analysis

iperf measured 7.71 Gbits/sec TCP bandwidth between h1 and h2. This high value is expected in a virtual Mininet network running on a single machine — packets travel over virtual interfaces on the same host OS, not real network hardware. Blocked hosts h3 and h4 cannot achieve any throughput because their packets are dropped at the switch level before reaching any destination.

8.3 Flow Table Analysis

The flow table showed 33 packets and 2450 bytes handled by the controller through the table-miss rule. After flow rules are installed for specific host pairs, future matching packets bypass the controller and are not counted in this statistic. This demonstrates efficient use of flow rules — the controller is only involved in the first packet of each new flow.

Metric	Value	Interpretation
h1 to h2 first ping latency	4.20 ms	packet_in round-trip to controller
h1 to h2 avg latency	1.578 ms	Low latency once flow rules installed
h1 to h2 throughput	7.71 Gbits/sec	High bandwidth for authorized hosts
h3 to h1 packet loss	100%	Blocking working perfectly
h4 to h2 packet loss	100%	Policy consistent across blocked hosts
Flow table packets	33 packets / 2450 bytes	All new flows handled by controller
Deny rule idle timeout	30 seconds	Flow table stays clean automatically
Allow rule idle timeout	60 seconds	Stable sessions for whitelisted hosts

9. Validation Summary

Test	Command	Expected	Actual Result	Status
Scenario A	h1 ping -c 3 h2	0% loss	0% loss	PASSED
Scenario B	h3 ping -c 3 h1	100% loss	100% loss	PASSED

Test	Command	Expected	Actual Result	Status
Regression	h4 ping -c 3 h2	100% loss	100% loss	PASSED
Throughput	iperf h1 h2	High throughput	7.71 Gbits/sec	PASSED
Flow table	ovs-ofctl dump-flows s1	Rules installed	priority=0, 33 packets	PASSED
Controller logs	Ryu terminal	BLOCKED messages	Visible for h3 and h4	PASSED
Wireshark	icmp filter	h1/h2 replies only	Confirmed	PASSED

10. Conclusion

This project successfully implemented an SDN-based access control system using Mininet and the Ryu OpenFlow controller. The system demonstrated all required aspects of the assignment:

- Correct controller-switch interaction via OpenFlow packet_in events and flow rule installation
- Effective MAC-based whitelist filtering with explicit match-action flow rules
- Consistent access control policy across all unauthorized hosts, verified by regression testing
- Measurable performance metrics confirming efficient forwarding for authorized hosts
- Complete proof of execution through ping results, iperf, flow table dumps, Wireshark captures, and Ryu controller logs

The use of priority levels (0, 5, 10) and idle timeouts (30s deny, 60s allow) ensured correct rule precedence and automatic flow table cleanup. The key insight demonstrated is that SDN allows network access control to be enforced entirely in software — adding or removing an authorized host requires only a one-line change to the whitelist in the controller, with no hardware reconfiguration needed.